Friend function:. A function that can access the private members of the class even though it is not the part of the class

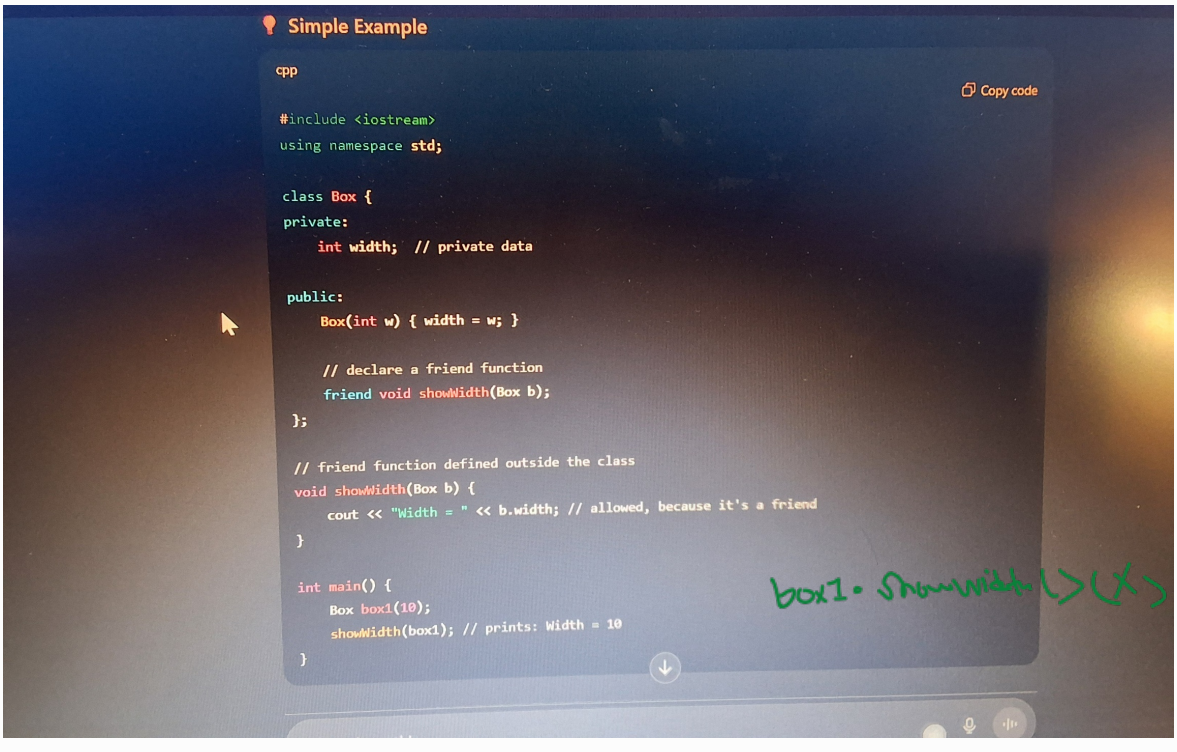
Syntax

friend returntype functionname (arguments);

It's a normal fuction (not part of any class)

Declared inside the class ∈

Defined outside the class ∈



It is not a member Function.

wrong.

till now only member functions could access private | protected data

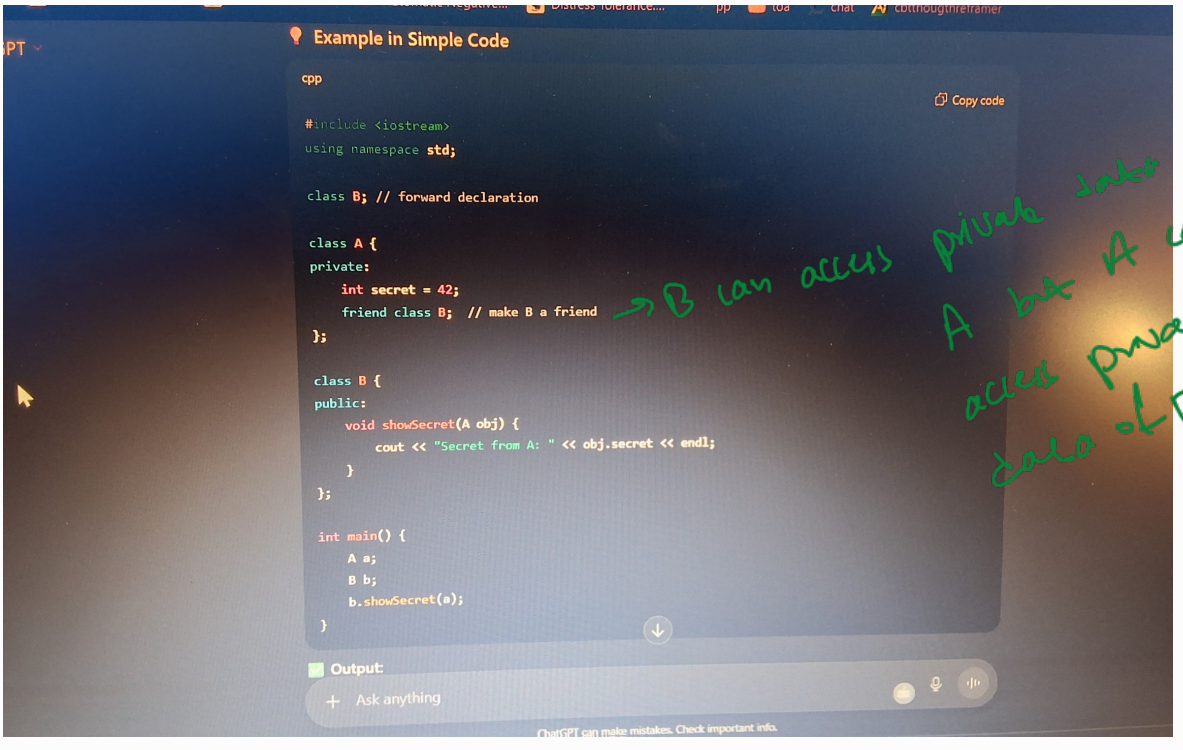
Friend class:. A special class that is allowed to see and use the private and protected data of another class

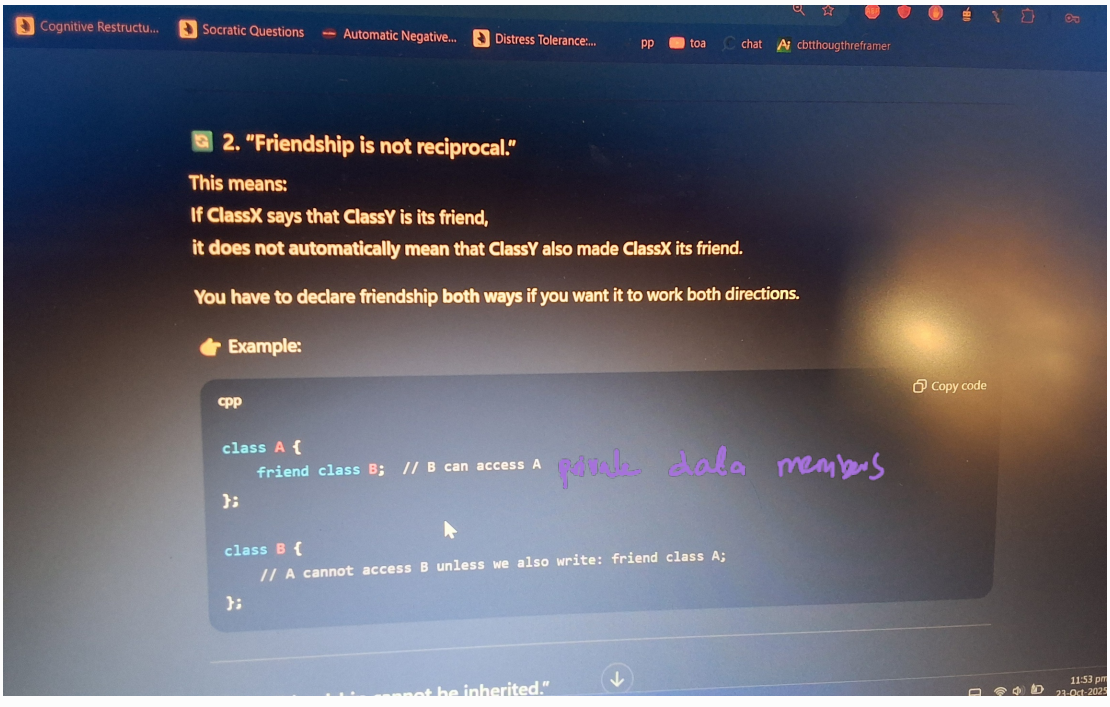
class S

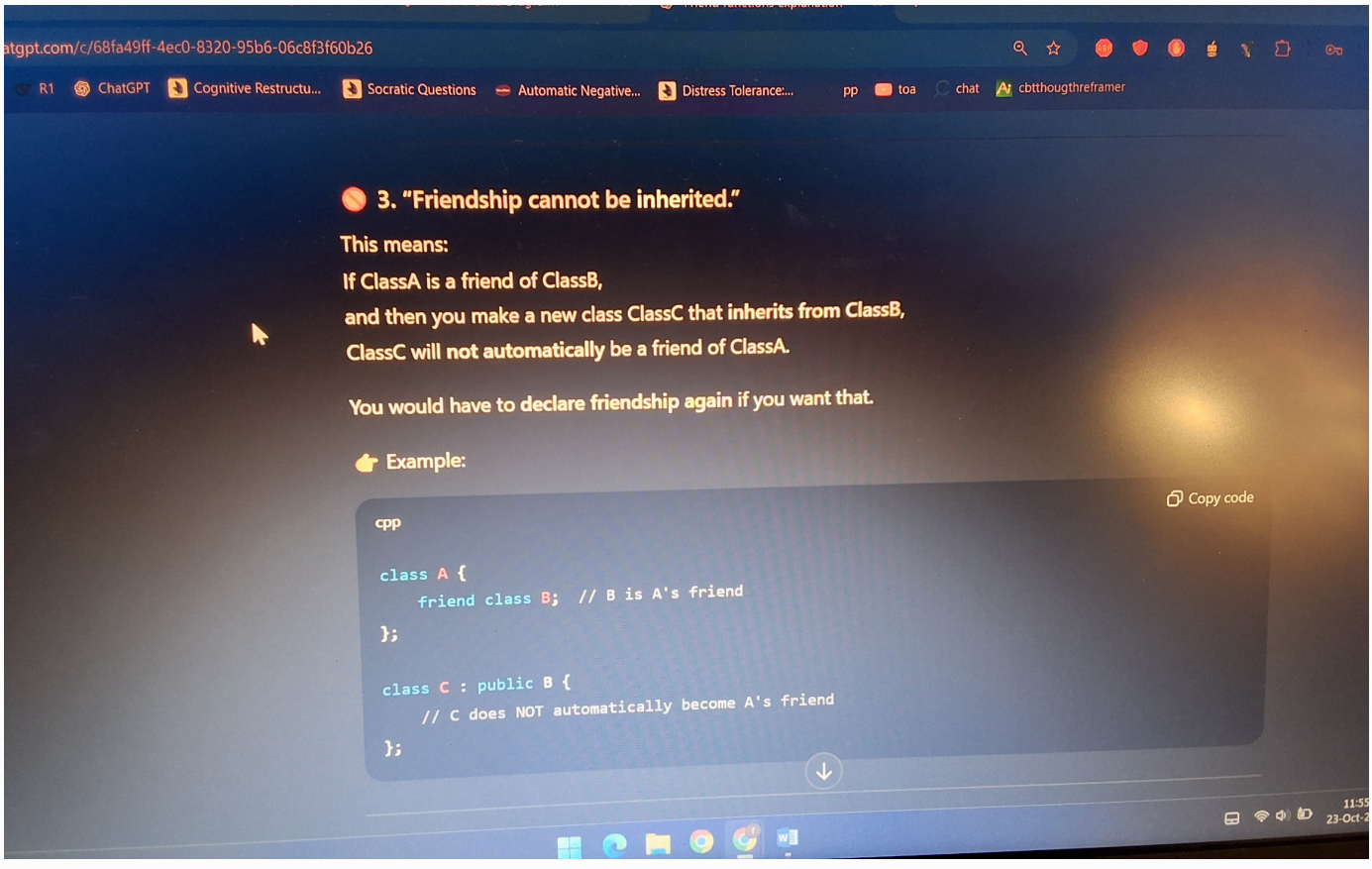
class P {
friend class S;
};

class S {
    // now S can access private and
}; protected members of P.

Example:

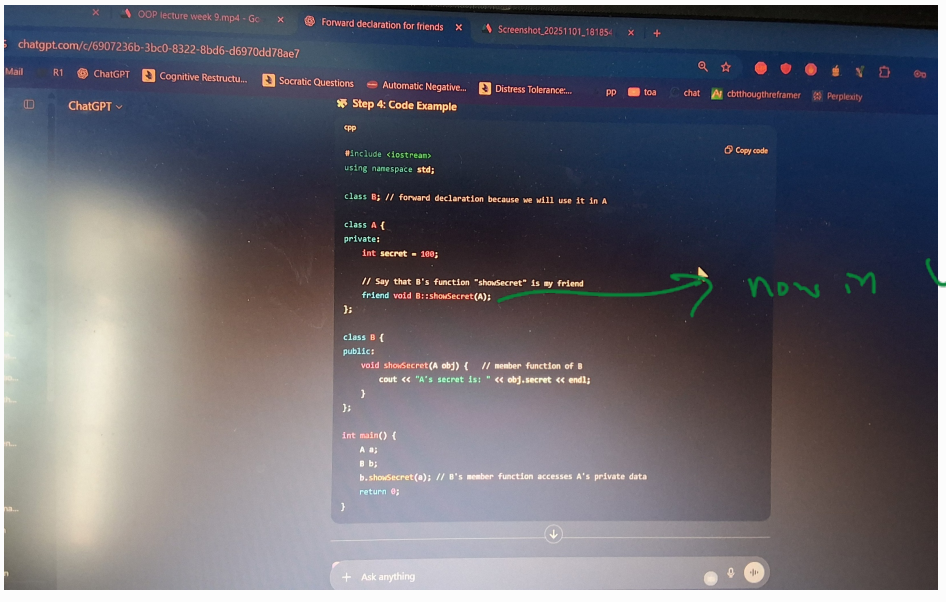






A member function of one class can be a friend of another class.

friend returntype Class name :: Function name (parameters);



now in class B only Show secret can see the private data.

Relationship b/w classes and objects.

1) Dependency ----> (use a)
2) Association ——> (use-a)
3) Aggregation ——◇ (has a)
4) composition ——◆ (whole part)
5) Inheritance ——▷ (is-a)

[] → Subscript notation refers to an object with [].

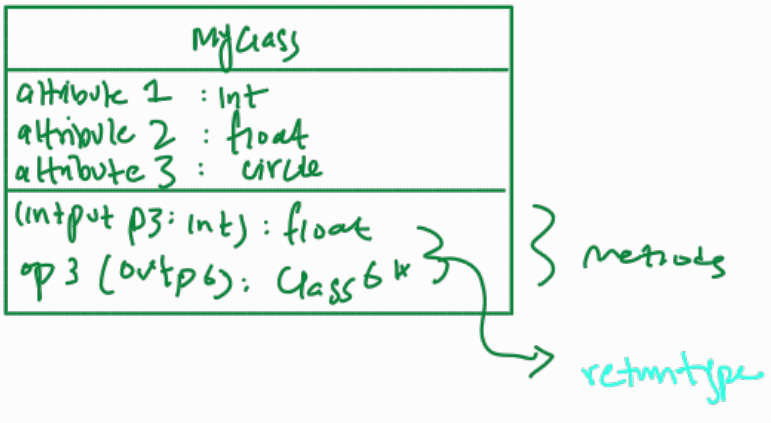
Object []

```
class numbers {
    int data[3] = {10, 20, 30};

    public
        int operator[](int index) {
            return data[index];
        }
};
```

In main
numbers[0] → 10

Class diagram



Association (uni bi) → objects may / may not know of each other.

1) one to one
2) one to many
3) many to many

Aggregation one to one
one to many

Composition → ownership involved
Strong relation.

1) one to one
2) one to many

} cardinality aka multiplicity.

Dependency

object created inside the function

Relationship among classes : Purpose To organize and structure classes in meaningful ways

→ To prompt reuseability of classes
→ To model real world accurately

Factors of Relationship

→ Strength
→ Lifetime dependency
→ Direction of relationship
→ nature of relationship e.g is-a

Relationship

1) Dependency (uses-a) one class needs another temporarly to perform a task
(weakest) of all relationship and exist only during the execution of a method

once the method finishes the relationship disappears.
→ one object doesnot know another object details.
→ Doesn't express cardinality (one-one e.g)

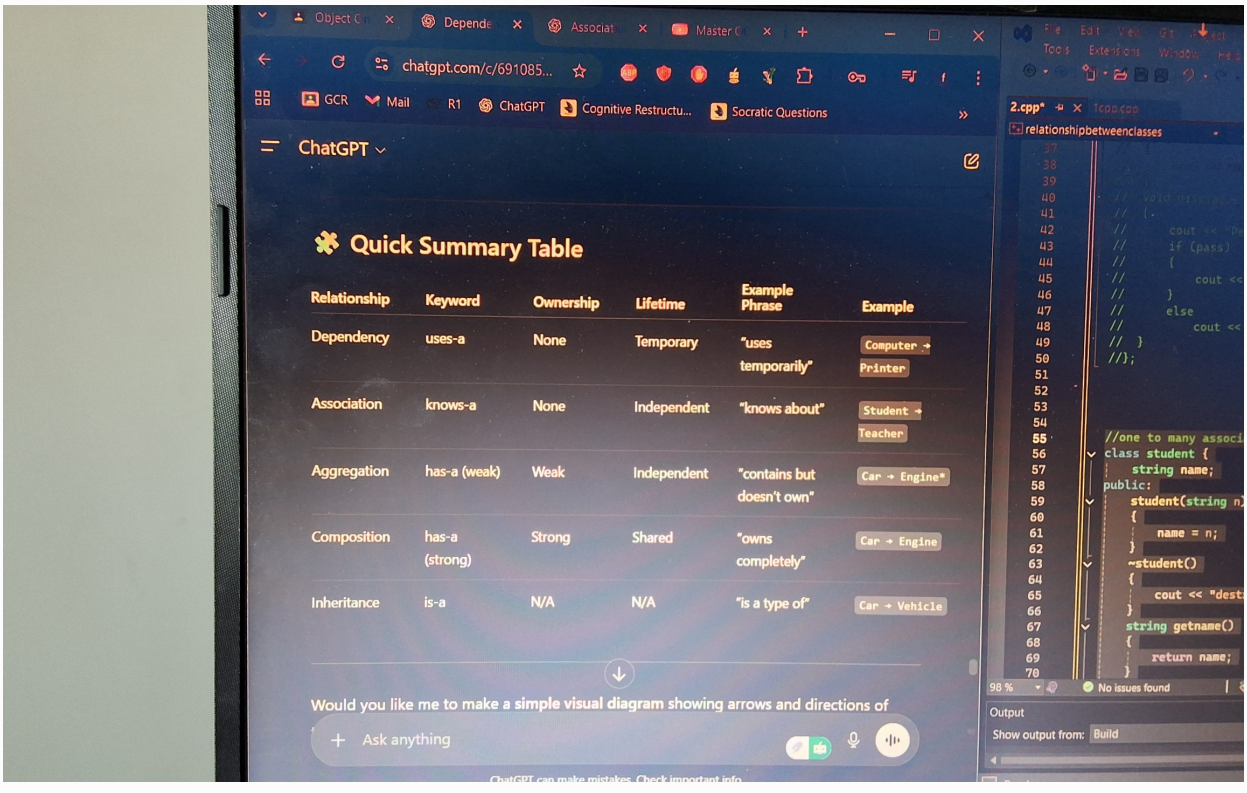
2) Association .. one class knows about another class and can interact but doesn't define who owns whom, classes exist seperately.
↓
Stronger form of dependency

("knows-a")

Purpose → allows objects to collaborate without ownership
→ Provide Real-world interaction.

→ avoid tight coupling

1) one-one association Passport ← person
2) one-many association Books ← library
3) many-many association many ← one
Many courses ↔ Students



Factors that affect relationships b/w classes

1) object life time → how long the object exist
2) ownership/control → which class owns the other object
3) Direction → who communicates with who
4) Multiplicity → how many objects are involved. One-One
5) Access → How one class accesses another e.g via pointer

note: Destructor runs in reverse order.

```
int main ()
{
    Student S1 ("Ali");
    Student S2 ("Sara");
    Student S3 ("Sir John");
}
```

as soon as goes out of main

S3
S2
S1
} → destructor runs like this.